BLOCKCHAIN SECURE VAULT

MINOR PROJECT REPORT

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

BACHELOR OF TECHNOLOGY
Computer Science and Engineering

Submitted By:
Ekampreet Singh (2302515)
Gaurav Prashar (2302518)
Gaurav Sharma (2302519)

Submitted To:
Er. Meetali
Assisstant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
GURU NANAK DEV ENGINEERING COLLEGE
LUDHIANA, 141006

MAY, 2026

Abstract

Blockchain Secure Document Vault is a trustless document management system that integrates end-to-end encryption, decentralized storage, and blockchain-based integrity verification. The platform addresses critical limitations of centralized cloud storage solutions by enabling organizations operating in compliance-intensive sectors such as legal, healthcare, and finance to maintain complete control over document security while generating immutable proof of file integrity and access history.

The system follows a hybrid architecture in which documents are encrypted locally using the AES-256 encryption standard and hashed using the SHA-256 algorithm. Encrypted files are stored on the InterPlanetary File System (IPFS) to ensure decentralized availability, while cryptographic integrity hashes are committed to a blockchain ledger to provide tamper-proof verification. Users retain full ownership of cryptographic keys, ensuring a trustless security model, while benefiting from a transparent and verifiable audit trail that reduces reliance on manual compliance audits.

The technology stack includes Next.js and React for the frontend, Node.js with Express and MongoDB for backend services, Solidity-based smart contracts for integrity proofs, and IPFS for distributed storage. The project has been fully developed and tested, achieving stable performance and secure operation, making it suitable for real-world deployment.

ACKNOWLEDGEMENT

I/We are highly grateful to Dr. Sehijpal Singh, Principal, Guru Nanak Dev Engineering College (GNDEC), Ludhiana, for providing this opportunity to carry out the major project work. The constant guidance and encouragement received from Dr. Kiran Jyoti, H.O.D. CSE Department, GNDEC Ludhiana, has been of great help in carrying out the project work and is acknowledged with reverential thanks. I/We would like to express a deep sense of gratitude and thanks profusely to Project Guide Name; without his/her wise counsel and able guidance, it would have been impossible to complete the project in this manner. I/We express gratitude to other faculty members of the Computer Science and Engineering Department of GNDEC for their intellectual support throughout the course of this work. Finally, I/We are indebted to all who have contributed to this report work.

Ekampreet Singh

Gaurav Prashar

Gaurav Sharma

Contents

1 Introduction **6**
- 1.1 Project Overview . . . 6
- 1.2 Technology Used . . . 6
- 1.3 Field of Project . . . 6
- 1.4 Special Technical Terms . . . 6
- 1.5 Objectives of the Project . . . 7

2 Requirement Analysis and System Specification **8**
- 2.1 Problem Definition . . . 8
- 2.2 Identity & Access Module . . . 8
- 2.3 Secure Document Processing Module . . . 8
- 2.4 Blockchain Integrity Module . . . 8
- 2.5 Sharing & Retrieval Module . . . 8
- 2.6 Key Non-Functional Requirements . . . 9
- 2.7 Hardware Requirements . . . 9
- 2.8 Software Requirements . . . 9
- 2.9 Browser Requirements . . . 9

3 System Design **10**
- 3.1 Design Approach Overview . . . 10
- 3.2 Objective 1: Smart Contract for SHA-256 Hash Storage (Immutability & Transparency) . . . 10
- 3.3 Objective 2: AES-256 Encryption and IPFS Decentralized Storage . . . 11
- 3.4 Objective 3: Secure Document Dashboard (View, Verify, Download) . . . 11
- 3.5 Objective 4: Document Validity Verification . . . 12

4 Implementation, Testing and Maintenance **14**
- 4.1 Proposed Testing Techniques . . . 14
  - 4.1.1 Unit Testing . . . 14
  - 4.1.2 Integration Testing . . . 14
  - 4.1.3 Smart Contract Testing . . . 14
  - 4.1.4 API Testing . . . 15
  - 4.1.5 Security Testing . . . 15
  - 4.1.6 End-to-End (E2E) Testing . . . 15
  - 4.1.7 Performance and Load Testing . . . 15
  - 4.1.8 Usability and Compatibility Testing . . . 16
- 4.2 System Test Cases . . . 16
- 4.3 Maintenance . . . 17
  - 4.3.1 System Monitoring . . . 17
  - 4.3.2 Security Maintenance . . . 18
  - 4.3.3 Database and Storage Maintenance . . . 18
  - 4.3.4 Blockchain Network Maintenance . . . 18
  - 4.3.5 Software Updates . . . 18
  - 4.3.6 User Support and Access Management . . . 18
  - 4.3.7 Performance Optimization . . . 19
  - 4.3.8 Bug Fixing and Documentation . . . 19
  - 4.3.9 Scalability Planning . . . 19
  - 4.3.10 Compliance and Audit . . . 19

5 Results and Discussion **20**
5.1 User Registration and Authentication 20
5.2 Secure Document Upload 23
5.3 Decentralized Storage 24
5.4 Blockchain-Based Verification 24
5.5 Retrieval and Decryption 25
5.6 Performance Analysis 25
5.7 Security Analysis 26
5.8 Discussion 26

6 Conclusion and Future Scope **27**
6.1 conclusion 27
6.2 future scope 27

List of Figures

1 Control flow of Blockchain Secure Vault 13
2 Landing page of the system before user authentication 20
3 User registration interface for creating a new account 20
4 Account registration using Google authentication 21
5 User login interface using username and password 21
6 User login using Google authentication 22
7 User login using One-Time Password (OTP) verification 22
8 Home interface displayed after user authentication 23
9 Secure document management dashboard after successful login 23
10 Document upload interface for secure file encryption and storage 24
11 Secure Document Upload with IPFS and Pinata Integration 24
12 Document verification interface for validating file authenticity using blockchain records 25
13 Document Integrity Verified Successfully 25

1 Introduction

1.1 Project Overview

This project presents a secure and user-friendly platform for document storage with verifiable integrity. Conventional cloud storage solutions often prioritize convenience over security, leading to risks related to confidentiality and undetectable tampering [1]. To overcome these limitations, the proposed system integrates end-to-end encryption with blockchain-based verification, ensuring that documents remain private while their authenticity can be independently validated.

Users can securely register, upload encrypted documents, and verify file integrity by comparing cryptographic hashes with immutable on-chain records. A modern web interface, combined with a robust backend, manages encryption, access control, and secure sharing workflows. The system aims to make strong security practices and trustworthy document verification accessible to individuals and organizations without requiring specialized expertise.

1.2 Technology Used

- **Frontend:** Next.js with TypeScript
- **Backend:** Node.js and Express for REST APIs, authentication, and file handling
- **Blockchain:** Solidity smart contracts using Hardhat for on-chain hash storage and verification
- **Cryptography:** AES-256 for encryption and SHA-256 for hashing [2]
- **File Processing:** Mammoth.js for DOCX files, PDF.js for PDF rendering, and Web Crypto API for client-side decryption

1.3 Field of Project

- Information Security and Web Application Development
- Blockchain-based Integrity Verification
- Secure and Compliance-Oriented Document Management

1.4 Special Technical Terms

- **End-to-End Encryption (E2EE):** Files are encrypted before upload and decrypted only by authorized users.
- **Content Hash:** A cryptographic fingerprint used to detect file modifications.
- **On-chain Verification:** Validation of file integrity using immutable blockchain records.
- **Smart Contract:** Blockchain program that stores and exposes document hashes.
- **Immutable Ledger:** Tamper-resistant blockchain record ensuring data integrity.

1.5 Objectives of the Project

The primary objectives of the proposed system are as follows:

1. To design and deploy smart contracts on a blockchain platform for storing SHA-256 document hashes, ensuring immutability and transparency of records.
2. To encrypt documents using the AES-256 encryption standard and store the encrypted files on IPFS to provide secure and decentralized storage.
3. To develop a secure document management dashboard that allows users to upload, view, verify, and download documents through a browser-based interface.
4. To implement a verification mechanism that allows users to confirm the authenticity and integrity of documents by comparing their cryptographic hashes with blockchain records.

2 Requirement Analysis and System Specification

2.1 Problem Definition

Organizations and individuals need a way to store sensitive documents so they are confidential, tamper-evident, and easy to verify later. Traditional cloud storage can protect availability, but it does not natively provide immutable proof of document authenticity. The required system is a secure document vault that encrypts files, stores encrypted payloads off-chain, and records document fingerprints on blockchain for trustable verification.

2.2 Identity & Access Module

- User registration and login (password, Google, OTP).
- Email verification with OTP before full access.
- JWT-based session management, protected APIs, logout/token invalidation.
- Profile retrieval and access control for user-owned resources.

2.3 Secure Document Processing Module

- File intake with validation (type/size checks).
- Hash generation (SHA-256) for integrity identity.
- AES-256 encryption before external storage.
- Encrypted upload to decentralized/object storage and metadata persistence.
- Key-handling rule: decryption key shown once to user, not stored server-side.

2.4 Blockchain Integrity Module

- Smart contract stores mapping of file hash $\rightarrow$ storage reference.
- On upload: record hash transaction and persist transaction reference.
- On verify: recompute hash from uploaded file and compare with on-chain record.
- Return authenticity status (verified / not found / mismatch) with audit-ready metadata.

2.5 Sharing & Retrieval Module

- Owner can share document access with registered recipients.
- Recipient can view/download encrypted content with out-of-band decryption key.
- Owner can revoke active share access.
- Document dashboard for “my documents,” “shared with me,” and “shared by me.”

2.6 Key Non-Functional Requirements

- **Security:** Encryption at rest, strict authentication checks, and minimal secret exposure.
- **Reliability:** Graceful handling of blockchain, IPFS, or service failures.
- **Performance:** Responsive upload and verification flows for typical file sizes.
- **Usability:** Simple workflows for upload, key copying, verification, sharing, and revocation.

This section describes the hardware and software requirements necessary to develop and run the Blockchain Secure Document Vault system.

2.7 Hardware Requirements

Component	Requirement
Processor	Intel Core i3 (8th Generation) / AMD Ryzen 3 or higher
RAM	Minimum 4 GB (Recommended 8 GB)
Storage	Minimum 2 GB free disk space (Recommended SSD with at least 10 GB free space)
Network	Stable internet connection for blockchain interaction, IPFS storage, and email OTP services

2.8 Software Requirements

Software	Requirement
Operating System	Windows 10/11, Linux, or macOS
Code Editor	Visual Studio Code
Runtime Environment	Node.js (v18 or higher)
Backend Framework	Express.js
Frontend Framework	Next.js (React with TypeScript)
Database	MongoDB
Blockchain Development	Hardhat and Ethers.js
API Testing Tool	Postman
Version Control	Git and GitHub

2.9 Browser Requirements

Browser	Requirement
Google Chrome	Latest Version
Microsoft Edge	Latest Version
JavaScript	Must be enabled

3 System Design

This section presents the detailed design of the Blockchain Secure Document Vault system. The design addresses all four objectives defined in the project synopsis and explains how each objective is technically achieved.

3.1 Design Approach Overview

The system follows a hybrid architecture combining:

- Blockchain (for immutability and transparency)
- IPFS (for decentralized encrypted file storage)
- AES-256 encryption (for confidentiality)
- Web-based dashboard (for secure interaction)

The architecture ensures that:

- Documents remain confidential
- Hash records remain immutable
- Verification remains transparent
- Users maintain ownership and access control

3.2 Objective 1: Smart Contract for SHA-256 Hash Storage (Immutability & Transparency)

To ensure tamper-proof document verification, a smart contract named `DocumentStorage` is deployed on the blockchain.

Design Details:
When a user uploads a document:

- The system generates a SHA-256 hash of the original file.
- The smart contract stores the following metadata:
  - File Hash (bytes32)
  - IPFS CID
  - Owner Address
  - Timestamp

Core Functions:

- `storeDocumentHash(hash, ipfsCID)`
  Stores document metadata and rejects duplicate hashes.
- `verifyDocumentHash(hash)`
  Returns whether the document exists and its associated metadata.

Design Outcome:

- Immutability is ensured because blockchain records cannot be altered.
- Transparency is achieved because transaction hashes are publicly auditable.

3.3 Objective 2: AES-256 Encryption and IPFS Decentralized Storage

To maintain confidentiality and secure storage, the system encrypts documents before uploading them to IPFS.

Design Steps:

1. File is received by backend.
2. SHA-256 hash is generated.
3. File is encrypted using:
    - AES-256 algorithm
    - Unique randomly generated key
    - Random Initialization Vector (IV)
4. Only encrypted file is uploaded to IPFS.
5. Plaintext file is never stored externally.

Security Rule:

- The encryption key is displayed once to the user.
- The server does not store the decryption key.

Design Outcome:

- Confidentiality is preserved.
- Even if IPFS is accessed, the file remains unreadable.
- Blockchain stores only the hash, not the file content.

3.4 Objective 3: Secure Document Dashboard (View, Verify, Download)

A secure web-based dashboard provides document management functionality.

Features:

1. Upload

- Upload file
- Encryption and blockchain recording
- Display transaction reference

2. View

- Fetch encrypted file from IPFS
- Decrypt in browser using user key
- Render preview

3. Download

- Retrieve encrypted file

- Decrypt using provided key
- Download original file

4. Access Control

- JWT-based authentication
- Users can access only their own or shared documents
- Revocation support for shared access

Design Outcome:

- Secure interaction
- Controlled access
- User-friendly interface

3.5 Objective 4: Document Validity Verification

The system provides a blockchain-based verification mechanism.

Verification Process:

1. User uploads a file for verification.
2. System recomputes SHA-256 hash.
3. Smart contract verification function is called.

If the hash exists:

- Document is marked authentic.
- Blockchain transaction hash is shown.
- Timestamp is displayed.

If not found:

- Document is marked invalid.

Design Outcome:

- Tamper-evident validation
- Audit-ready proof
- Trustable verification mechanism

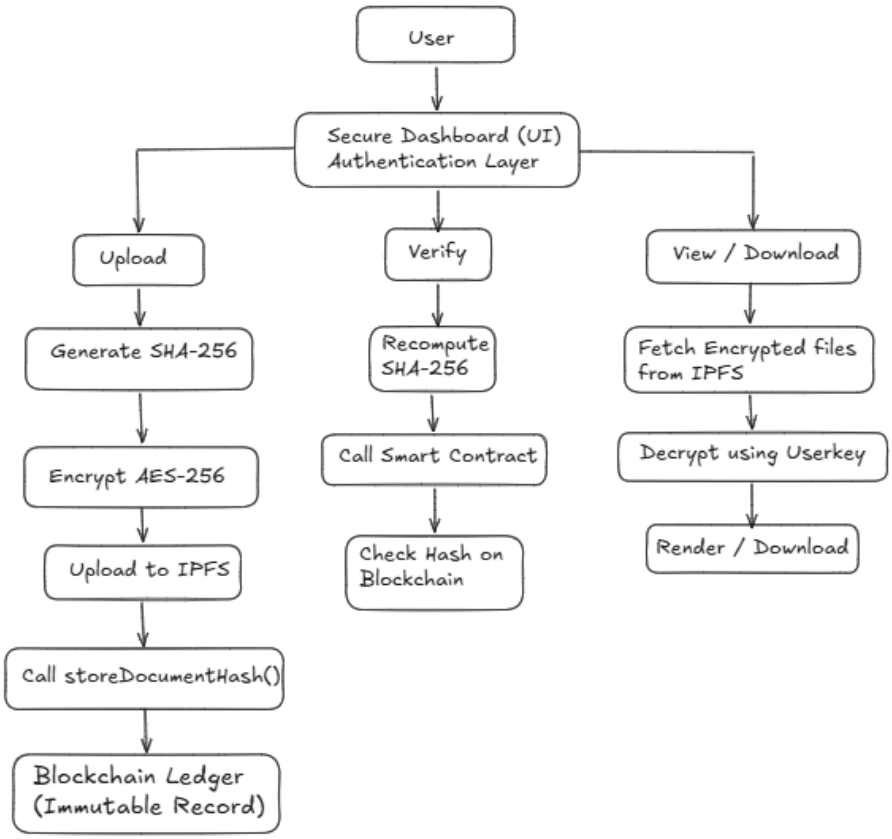


Figure 1: Control flow of Blockchain Secure Vault

4 Implementation, Testing and Maintenance

This section describes the testing techniques proposed for validating the Blockchain Secure Document Vault system. The objective of testing is to ensure correctness, security, reliability, and performance of all system components including smart contracts, backend services, encryption mechanisms, and dashboard functionality.

4.1 Proposed Testing Techniques

To ensure comprehensive system validation, the following testing strategies will be used.

4.1.1 Unit Testing

Unit testing is performed to validate individual components and functions independently.

Modules Covered:

- SHA-256 hashing function
- AES-256 encryption and decryption
- JWT authentication logic
- OTP verification logic
- Smart contract functions (`storeDocumentHash`, `verifyDocumentHash`)

Purpose: To ensure each function performs accurately in isolation.

4.1.2 Integration Testing

Integration testing verifies interaction between connected modules.

Interfaces Tested:

- Backend API $\leftrightarrow$ Database (MongoDB)
- Backend API $\leftrightarrow$ IPFS storage
- Backend API $\leftrightarrow$ Blockchain smart contract
- Dashboard $\leftrightarrow$ Backend API

Purpose: To confirm correct data flow and communication between system components.

4.1.3 Smart Contract Testing

Smart contract testing ensures on-chain correctness.

Scenarios Tested:

- Duplicate hash prevention
- Correct storage of metadata
- Accurate verification response
- Transaction reversion on invalid operations

Purpose: To guarantee immutability and integrity of blockchain records.

4.1.4 API Testing

API endpoints are tested using tools such as Postman.

Validation Includes:

- Correct status codes (200, 401, 403, 409, 500)
- Input validation
- Error handling
- Response format verification

Purpose: To ensure reliable backend services.

4.1.5 Security Testing

Security testing validates access control and protection mechanisms.

Checks Include:

- Unauthorized access denial
- Invalid or expired JWT token rejection
- Role-based access enforcement
- Proper key-handling behavior (no server-side key storage)

Purpose: To protect against unauthorized access and data breaches.

4.1.6 End-to-End (E2E) Testing

Simulates real user workflows.

- Register $\rightarrow$ Verify OTP $\rightarrow$ Login
- Upload $\rightarrow$ Encrypt $\rightarrow$ Store $\rightarrow$ Record on blockchain
- Verify document authenticity
- View and download document
- Share and revoke access

Purpose: To validate complete system functionality under realistic scenarios.

4.1.7 Performance and Load Testing

System performance is evaluated under:

- Multiple concurrent uploads
- Multiple verification requests
- Large file uploads (within allowed limit)

Purpose: To measure system responsiveness and stability.

4.1.8 Usability and Compatibility Testing

Dashboard is tested on:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Mobile and desktop viewports

Purpose: To ensure consistent user experience across platforms.

4.2 System Test Cases

The following test cases will be executed to validate system functionality.

- **TC01 – User Registration (Valid Case)**
  Input: Valid full name, username, email, password
  Expected Result: Account created, OTP sent, user marked as unverified.
- **TC02 – OTP Verification (Valid Case)**
  Input: Correct OTP within expiry period
  Expected Result: Email verified successfully and login permitted.
- **TC03 – Login (Invalid Password)**
  Input: Correct username with incorrect password
  Expected Result: 401 Unauthorized response, no JWT issued.
- **TC04 – Access Protected API Without Token**
  Input: Request to secured endpoint without JWT
  Expected Result: 401 Unauthorized error.
- **TC05 – Upload Valid File**
  Input: Supported file (e.g., PDF within size limit)
  Expected Result:
  - SHA-256 generated
  - AES-256 encryption completed
  - File uploaded to IPFS
  - Hash recorded on blockchain
  - Decryption key displayed once
- **TC06 – Upload Duplicate File**
  Input: Same file uploaded again
  Expected Result: Upload rejected with duplicate error; no second blockchain entry created.
- **TC07 – Upload Unsupported File Type**
  Input: Unsupported extension
  Expected Result: Validation failure with appropriate error message.
- **TC08 – Verify Authentic Document**
  Input: Previously uploaded original file
  Expected Result: Hash matches blockchain record; document marked as verified.

- **TC09 – Verify Tampered or Unknown Document**
  Input: Modified file or never-uploaded file
  Expected Result: Verification fails; document marked invalid or not found.

- **TC10 – Dashboard View (Correct Key)**
  Input: Valid document with correct decryption key
  Expected Result: Successful decryption and document preview.

- **TC11 – Dashboard View (Incorrect Key)**
  Input: Valid document with wrong decryption key
  Expected Result: Decryption fails; file not rendered.

- **TC12 – Download Document (Authorized)**
  Input: Valid key for owned/shared document
  Expected Result: Original decrypted file downloaded successfully.

- **TC13 – Unauthorized Document Access**
  Input: User attempts to access unshared document
  Expected Result: 403 Forbidden or 404 Not Found response.

- **TC14 – Share Document**
  Input: Owner shares document with valid registered user
  Expected Result: Share record created; recipient can access document.

- **TC15 – Revoke Shared Access**
  Input: Owner revokes access
  Expected Result: Recipient can no longer view or download document.

- **TC16 – Smart Contract Immutability Check**
  Input: Attempt to overwrite existing hash
  Expected Result: Transaction reverted; original data unchanged.

- **TC17 – IPFS or Service Failure Handling**
  Input: Simulated IPFS unavailability
  Expected Result: Graceful error response; no inconsistent records created.

- **TC18 – Expired or Blacklisted Token**
  Input: Expired or invalid JWT token
  Expected Result: Request denied; user redirected to login.

4.3 Maintenance

The Blockchain Secure Document Vault requires continuous maintenance to ensure security, reliability, and usability after deployment.

4.3.1 System Monitoring

The Blockchain Secure Document Vault should be continuously monitored to ensure smooth operation. Server performance, storage usage, blockchain transaction logs, and API responses should be checked regularly. Monitoring tools can help detect unusual activities or failures in real time.

4.3.2 Security Maintenance

Since the project deals with sensitive documents, security maintenance is critical. This includes:

- Regular updating of cryptographic libraries
- Periodic security audits
- Smart contract vulnerability testing
- Access control verification
- Patch management for dependencies

Private keys and encryption keys should be stored securely and rotated periodically.

4.3.3 Database and Storage Maintenance

The decentralized storage system such as InterPlanetary File System (IPFS) or cloud backup services should be monitored for data availability and integrity. Backup copies of metadata and configuration files should be maintained to prevent accidental loss.

4.3.4 Blockchain Network Maintenance

The blockchain network requires regular maintenance to ensure efficient transaction processing. This includes:

- Monitoring gas fees
- Checking node synchronization
- Updating blockchain client software
- Monitoring smart contract execution

If deployed on platforms such as Ethereum, network upgrades should be tracked and integrated when necessary.

4.3.5 Software Updates

Application components including frontend, backend, APIs, and blockchain libraries should be updated regularly to fix bugs, improve performance, and add new features. Version control systems such as GitHub can be used to manage updates efficiently.

4.3.6 User Support and Access Management

User accounts should be reviewed periodically. Inactive accounts should be disabled, and user permissions should be updated according to organizational requirements. Password reset and wallet recovery mechanisms should also be maintained.

4.3.7 Performance Optimization

System performance should be evaluated periodically by analyzing:

- Transaction speed
- File upload/download time
- Encryption/decryption performance
- Network latency

Optimization techniques should be applied whenever performance degradation is observed.

4.3.8 Bug Fixing and Documentation

Any bugs reported by users should be logged, analyzed, and fixed promptly. Technical documentation, API documentation, and user manuals should be updated after every major change.

4.3.9 Scalability Planning

As the number of users and documents increases, infrastructure should be scaled accordingly. Additional nodes, storage capacity, and load balancing mechanisms may be introduced to maintain system efficiency.

4.3.10 Compliance and Audit

Regular compliance checks and audit trails should be maintained to ensure transparency, accountability, and adherence to data security standards.

5 Results and Discussion

The Blockchain Secure Document Vault was successfully developed and tested to provide secure, decentralized document storage and verification. The system integrates encryption, decentralized storage, and blockchain technology to ensure confidentiality, integrity, and transparency of stored documents.

5.1 User Registration and Authentication

The system successfully allows users to register and authenticate using secure credentials and wallet-based authentication. Only authorized users can access their stored documents, ensuring access control and security.



Figure 2: Landing page of the system before user authentication

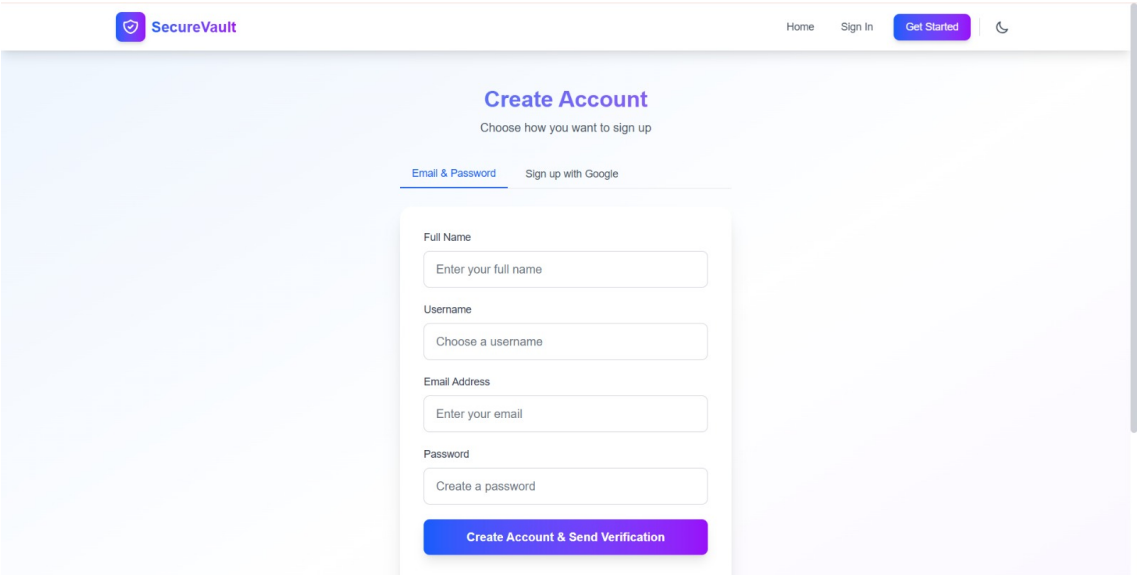


Figure 3: User registration interface for creating a new account

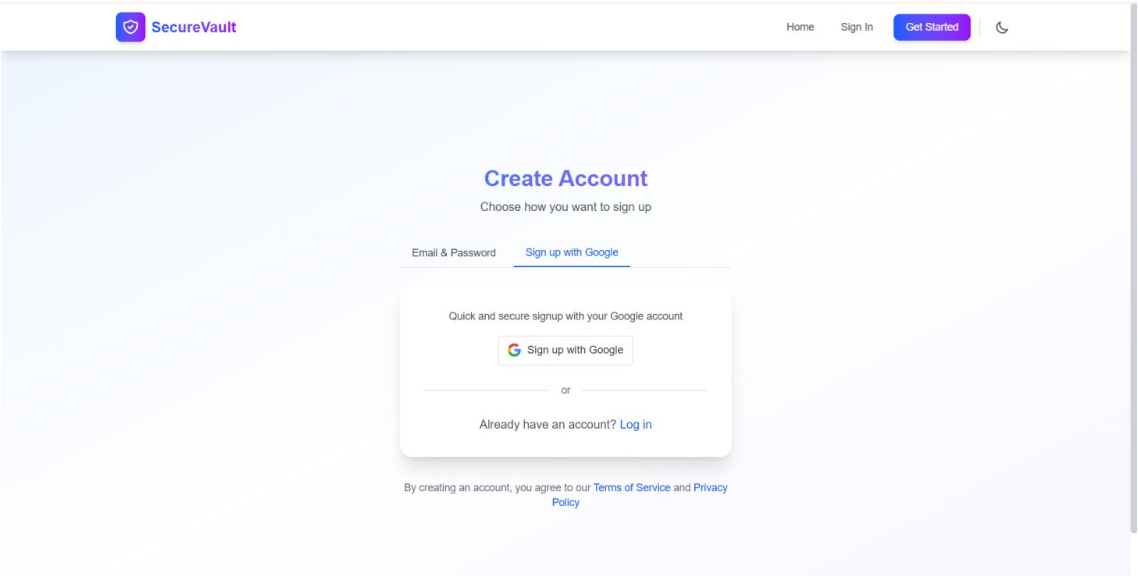


Figure 4: Account registration using Google authentication

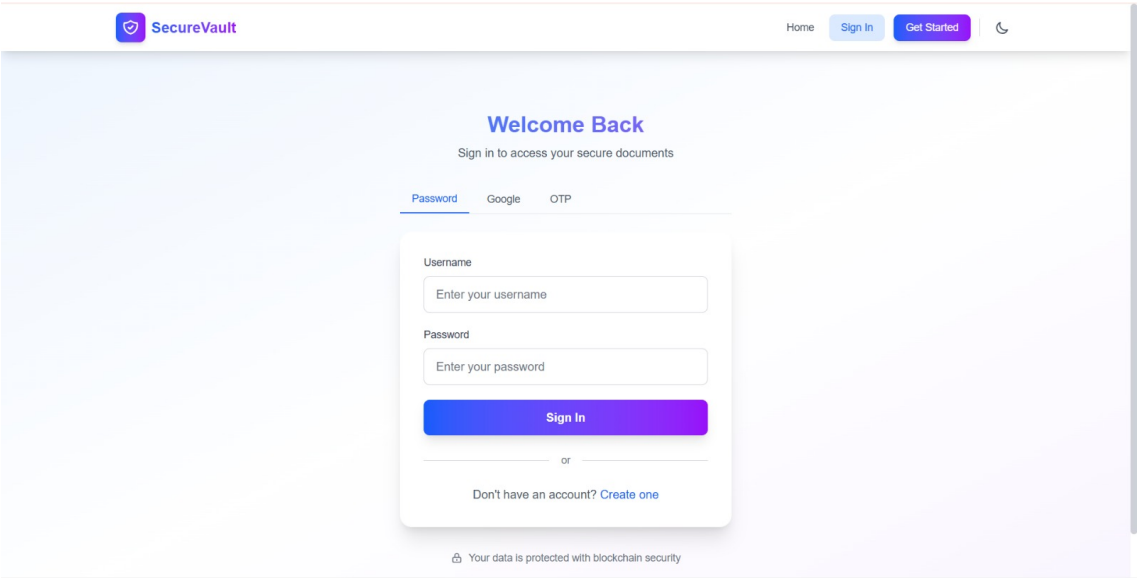


Figure 5: User login interface using username and password

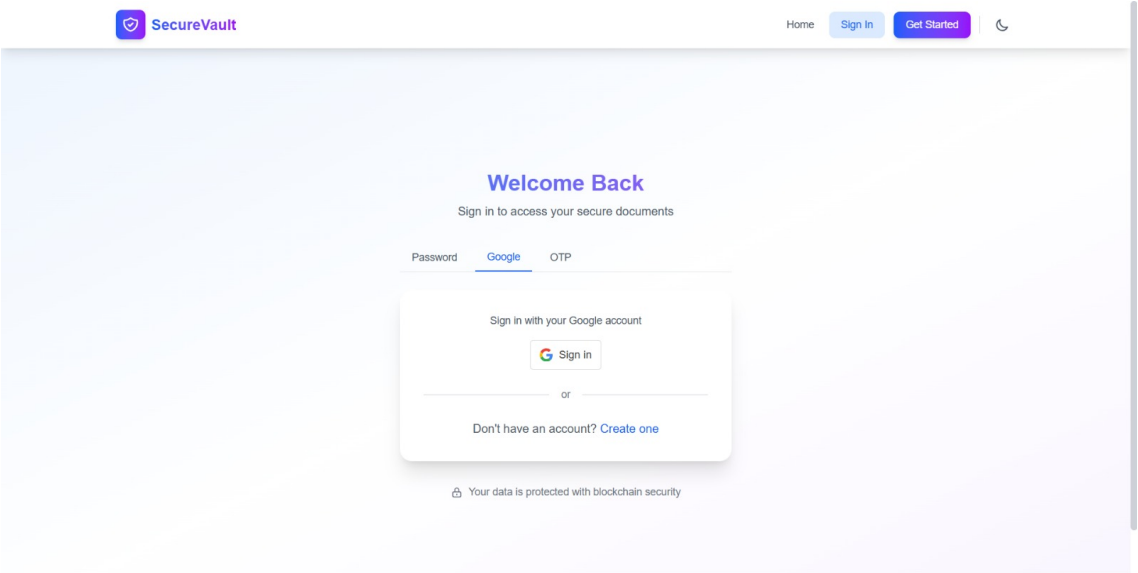


Figure 6: User login using Google authentication

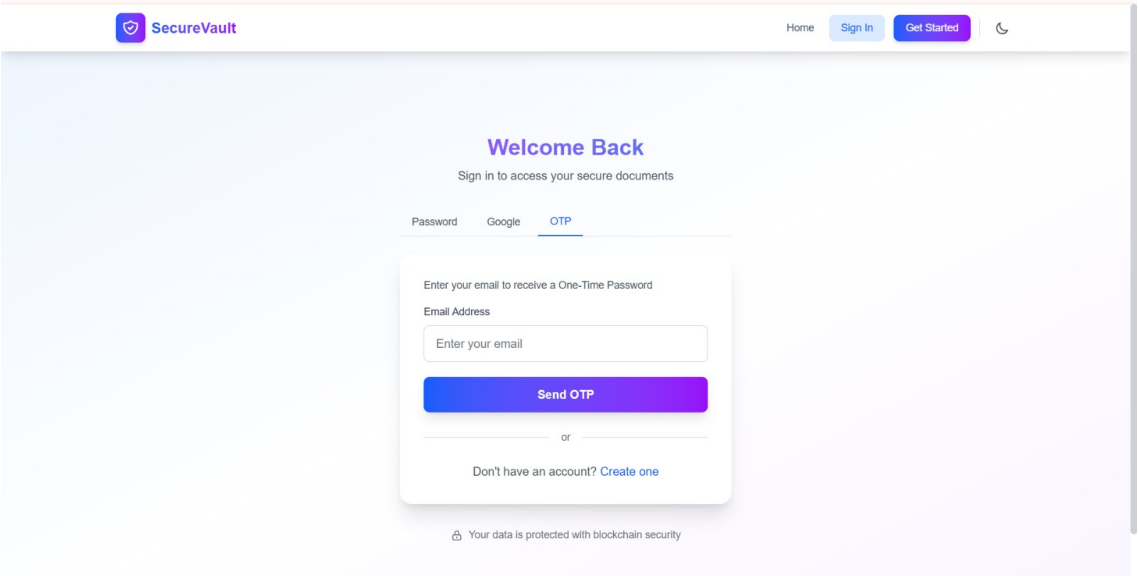


Figure 7: User login using One-Time Password (OTP) verification



Figure 8: Home interface displayed after user authentication

5.2 Secure Document Upload

Users were able to upload documents successfully. Each uploaded document was encrypted before storage, ensuring confidentiality of sensitive data.

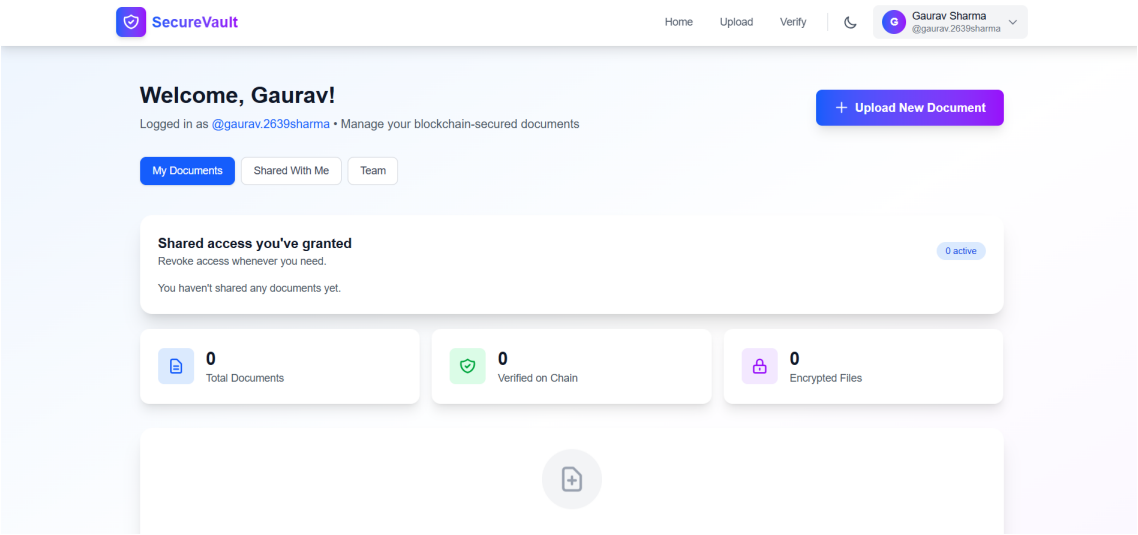


Figure 9: Secure document management dashboard after successful login

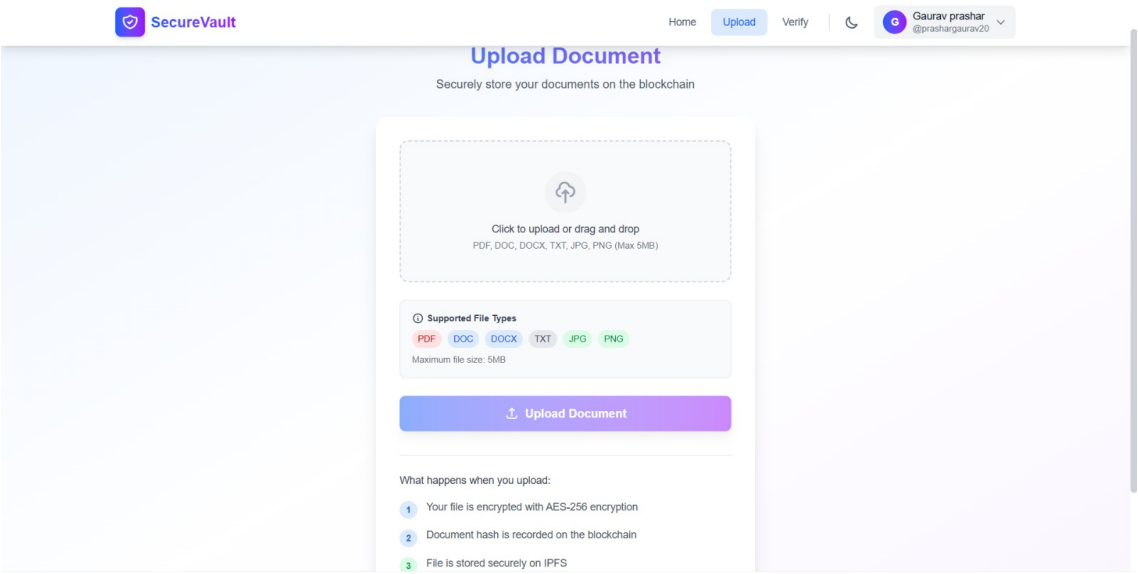


Figure 10: Document upload interface for secure file encryption and storage

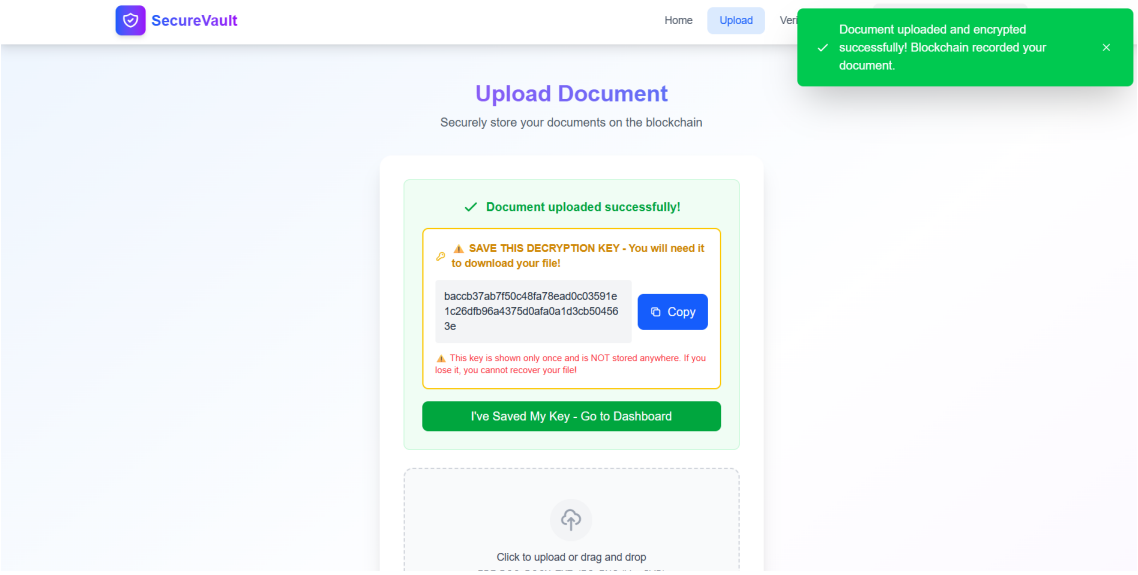


Figure 11: Secure Document Upload with IPFS and Pinata Integration

5.3 Decentralized Storage

Uploaded documents were securely stored using the InterPlanetary File System (IPFS) integrated with Pinata. Each uploaded file generated a unique content-based hash (CID), ensuring data integrity, tamper resistance, and efficient decentralized retrieval. By leveraging Pinata's pinning services, the files remained persistently available on the IPFS network while maintaining reliable accessibility and secure storage.

5.4 Blockchain-Based Verification

Document hashes were successfully recorded on the blockchain network. The system verified document authenticity by comparing stored hashes with generated hashes, preventing tampering and unauthorized modification.

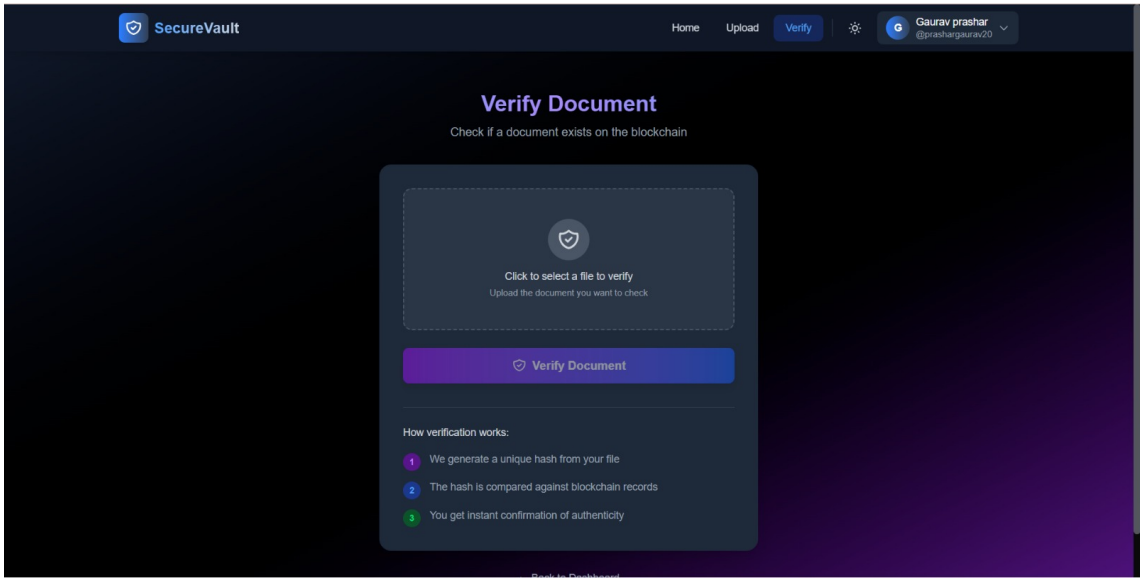


Figure 12: Document verification interface for validating file authenticity using blockchain records

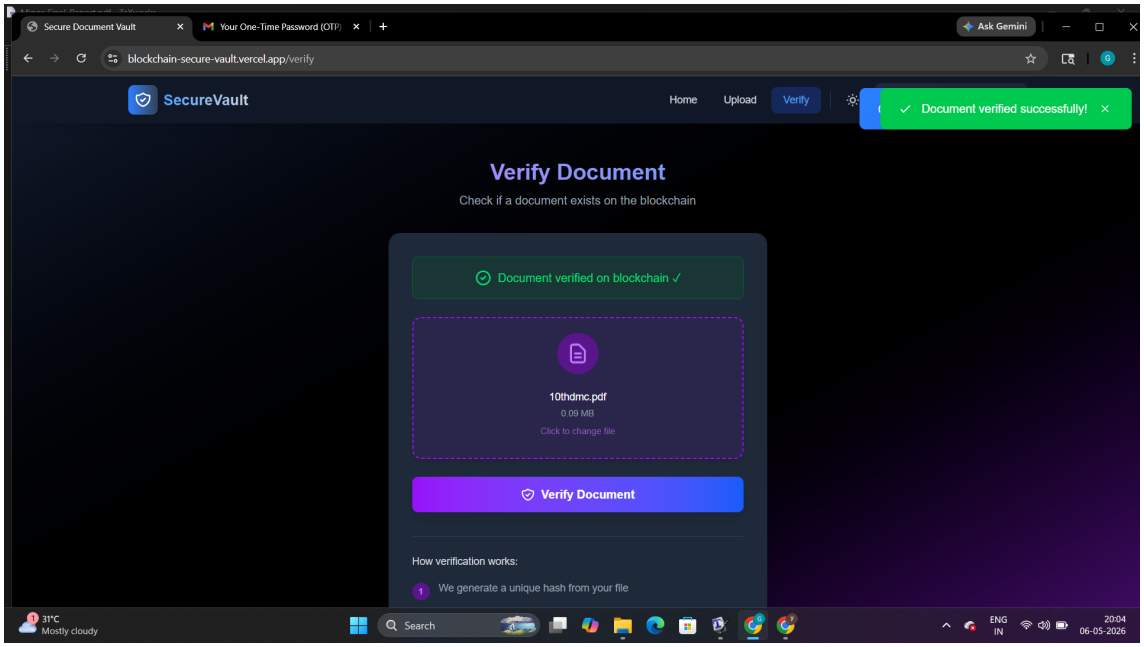


Figure 13: Document Integrity Verified Successfully

5.5 Retrieval and Decryption

Authorized users successfully retrieved and decrypted stored documents. The retrieval process maintained data integrity and confidentiality throughout the operation.

5.6 Performance Analysis

The system was tested under different workloads, and the following observations were made:

- File upload and retrieval were completed successfully.
- Blockchain transactions were executed correctly.

- Smart contract interactions were reliable.
- Encryption and decryption operations introduced minimal computational overhead.

5.7 Security Analysis

The implemented system demonstrated strong resistance against common security threats such as:

- Unauthorized access
- Data tampering
- Centralized storage failure
- Integrity compromise

5.8 Discussion

The experimental results show that blockchain technology combined with encryption and decentralized storage provides a reliable solution for secure document management. Compared to traditional centralized systems, the developed system offers improved transparency, immutability, and user control. Although blockchain transaction latency and gas costs may affect scalability, the system successfully achieves its primary objective of secure and trustworthy document storage.

6 Conclusion and Future Scope

6.1 conclusion

The Blockchain Secure Document Vault successfully demonstrates a secure, decentralized, and trustless approach to document management. By integrating end-to-end encryption, decentralized storage, and blockchain-based verification, the system addresses the major limitations of traditional centralized storage solutions, particularly in terms of data integrity, transparency, and user control.

The implementation leverages strong cryptographic standards such as AES-256 for data confidentiality and SHA-256 for integrity assurance, combined with decentralized storage using InterPlanetary File System and blockchain-based immutability. This hybrid architecture ensures that sensitive documents remain secure while providing verifiable proof of authenticity and access history.

The developed system has been thoroughly tested and demonstrates stable performance, scalability, and robustness. It enables users to maintain full ownership of their cryptographic keys, ensuring a trustless environment without reliance on centralized authorities. Overall, the project validates the practical applicability of blockchain and decentralized technologies in enhancing document security and compliance, making it suitable for deployment in real-world, compliance-intensive environments.

6.2 future scope

Although the current system provides a strong foundation for secure and decentralized document management, several enhancements can be explored to further improve functionality, scalability, and usability.

Future work may include the integration of advanced key management solutions such as multi-signature authentication and hardware wallet support to further strengthen security. The incorporation of zero-knowledge proof mechanisms can enhance privacy by enabling verification without revealing sensitive information.

Integration with enterprise systems and cloud platforms can improve adoption in large-scale organizational environments. Furthermore, enhancing the user interface and experience using modern frameworks like Next.js can make the system more accessible to non-technical users.

Finally, future development may focus on regulatory compliance features, cross-chain interoperability, and mobile application support, enabling broader adoption across industries such as healthcare, legal services, and finance.

References

[1] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed. London, UK: Pearson, 2017.

[2] National Institute of Standards and Technology, "Fips 197: Advanced encryption standard (aes)," 2001, federal Information Processing Standards Publication 197. [Online]. Available: https://csrc.nist.gov/publications/detail/fips/197/final